

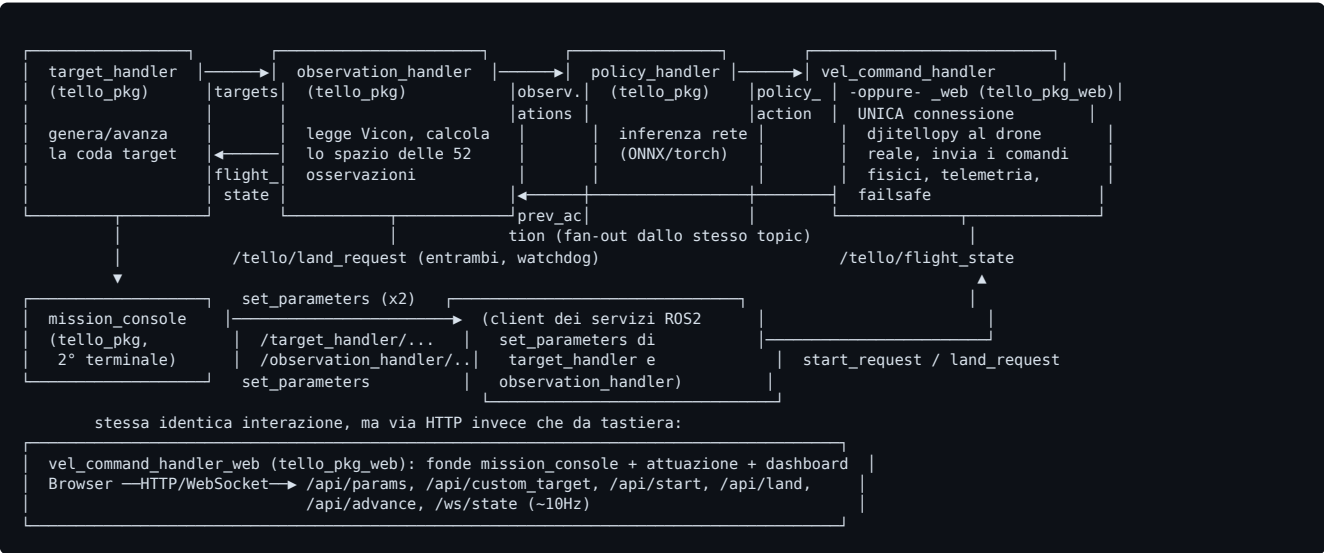
Architettura ROS2 — Tello Position Controller

Nodi, topic, servizi, frequenze di calcolo/pubblicazione — pipeline modulare (Percezione / Policy / Attuazione / Target)
Progetto: ROS2 NODE — pacchetti tello_pkg, tello_pkg_web, tello_test

1. Panoramica dell'architettura

Il sistema è organizzato in **quattro responsabilità separate**, ciascuna in un nodo ROS2 indipendente, che comunicano esclusivamente tramite topic/servizi — nessun nodo condivide memoria con un altro. Questa pipeline modulare esiste in **due varianti** che condividono gli stessi tre nodi "cervello" (target/osservazione/policy) e differiscono solo nel nodo di attuazione/controllo:

- **Versione da terminale** (tello_pkg): attuazione in vel_command_handler, controllo interattivo da un secondo terminale con mission_console.
- **Versione con interfaccia web** (tello_pkg_web): attuazione in vel_command_handler_web, che assorbe ANCHE il ruolo di mission_console (stessi servizi ROS2, esposti via endpoint HTTP invece che da wizard testuale) e aggiunge una dashboard 3D con WebSocket.



Nota sui dati "Vicon": `/vicon/tello_42_boosted/tello_42_boosted` e `/vicon/aruco42/aruco42` non sono pubblicati da nessuno dei nodi descritti in questo documento: arrivano da un **bridge Vicon esterno** (motion capture del laboratorio), a una frequenza decisa da quel sistema (tipicamente 100+ Hz), non da questo progetto. I nodi qui sotto sono tutti *subscriber* di quei due topic, mai publisher.

2. Tabella riepilogativa di tutti i topic

Topic	Tipo messaggio	QoS	Publisher	Subscriber	Frequenza	Contenuto
/vicon/tello_42_boosted/tello_42_boosted	geometry_msgs/PoseStamped	default (10)	bridge Vicon esterno	target_handler, observation_handler, vel_command_handler_web, takeoff_land.py, read_sensors.py, position_controller_VICON_VERSION.py, position_controller_web.py	event-driven (rate del Vicon)	Posa (quaternione, misura, Fonte, posizione)
/vicon/aruco42/aruco42	geometry_msgs/PoseStamped	default (10)	bridge Vicon esterno	target_handler (solo se target_mode=aruco_target), vel_command_handler_web (dashboard, sempre)	event-driven	Posa di (subject, la posizione, target f)
targets	std_msgs/Float32MultiArray (17 elementi)	reliable + transient_local , depth 1	target_handler	observation_handler, vel_command_handler_web (dashboard)	25 Hz	Codifica v[0]=wp, (4×xyz valori). subscriber subito
observations	std_msgs/Float32MultiArray (52 elementi)	default (10)	observation_handler	policy_handler	25 Hz	Vettore complesso sin/cos ang_vel prev_a waypoints dof_m
/tello/policy_action	geometry_msgs/Twist	default (10)	policy_handler	observation_handler (come prev_action), vel_command_handler oppure vel_command_handler_web (attuazione)	25 Hz (solo se obs fresca)	Azione clamp scaling sicurezza sull'azimuth angula
/tello/flight_state	std_msgs/Bool	reliable + transient_local , depth 1	vel_command_handler oppure vel_command_handler_web	target_handler (cattura target "hover" al fronte di salita)	event-driven (ad ogni transizione)	True quando considerato dopo t False c dell'attu
/tello/land_request	std_msgs/Empty	default (10)	target_handler (num_queues raggiunto), observation_handler (Vicon perso >3s consecutivi)	vel_command_handler oppure vel_command_handler_web	event-driven	Richiesta "esterno Ctrl+C, qualun emergenza
/tello/start_request	std_msgs/Empty	default (10)	mission_console (comando 'start'/s')	vel_command_handler oppure vel_command_handler_web	event-driven	Cancel modo f takeoff web il l lo stes: diretta dal top resta a
/target_handler/advance	std_msgs/Empty	default (10)	mission_console (INVIO), vel_command_handler_web (endpoint /api/advance)	target_handler	event-driven	Avanzamento prossim (solo t singola advanc

3. Servizi ROS2 (parametri a runtime)

Servizio	Tipo	Server	Client	Parametri gestiti
/target_handler/set_parameters	rcl_interfaces/SetParameters	target_handler (automatico via add_on_set_parameters_callback)	mission_console, vel_command_handler_web	target_mode, advance_mode, num_queues, custom_target_x/y/z/yaw
/observation_handler/set_parameters	rcl_interfaces/SetParameters	observation_handler	mission_console, vel_command_handler_web	dof_mask_mode

Ogni chiamata applica il parametro **immediatamente** (non al giro successivo): il callback rifiuta valori non validi (es. target_mode sconosciuto) o fuori dai limiti stanza (custom_target_x/y/z, validati contro ROOM_MIN/ROOM_MAX) restituendo `successful=False` con un motivo testuale.

4. Tabella riepilogativa delle frequenze

Nodo	Timer / ciclo	Periodo	Frequenza	Cosa fa ad ogni tick
target_handler	control_loop	0.04 s	25 Hz	eventuale refresh target (solo aruco_target), auto-advance (se convergenza tenuta per 1.2s), pubblica targets
observation_handler	control_loop	0.04 s	25 Hz	integrale d'errore, costruzione vettore 52D, pubblica observations (se pose fresca e targets ricevuti)
policy_handler	control_loop	0.04 s	25 Hz	forward pass rete (normalizzazione + MLP 256-128-64 ELU), pubblica /tello/policy_action (se obs age < 0.2s)
vel_command_handler / _web	watchdog_timer	0.2 s	5 Hz	se ultima policy_action > 0.2s, forza hover (rc a zero)
vel_command_handler / _web	telemetry_timer	1.0 s	1 Hz	poll stato djitellopy (batteria/quota), failsafe batteria <15%, escalation se stato fermo >3s
vel_command_handler	status_timer	1.0 s	1 Hz	log di stato a terminale (flight_ready, batteria, quota, età ultima azione)
vel_command_handler_web	state_timer	0.1 s	10 Hz	costruisce lo snapshot di stato broadcastato via WebSocket /ws/state
policy_handler	OBS_TIMEOUT_S	0.2 s	soglia	oltre questa età, l'osservazione è considerata scaduta: nessuna azione pubblicata
observation_handler	POSE_TIMEOUT_S	0.5 s	soglia	oltre questa età, la posa Vicon è scaduta: nessuna osservazione pubblicata
observation_handler	POSE_LOST_LAND_TIMEOUT_S	3.0 s	soglia	Vicon assente CONTINUATIVAMENTE oltre questo: pubblica /tello/land_request
vel_command_handler / _web	TELLO_LOST_LAND_TIMEOUT_S	3.0 s	soglia	telemetria drone ferma oltre questo: atterraggio automatico

5. Nodi della pipeline modulare — `tello_pkg` (versione terminale)

target_handler `tello_pkg`

Responsabilità: possiede tutta la logica di generazione/avanzamento del TARGET. Non tocca mai `djitellopy`, nessuna connessione al drone.

Parametri (runtime, via `set_parameters`)

- `target_mode`: singolo | variabile | custom | hover | `aruco_target` (default variabile)
- `advance_mode`: manual | auto (default manual)
- `num_queues`: intero, ≤ 0 = infinito (default -1)
- `custom_target_x/y/z/yaw`: coordinate del target fisso (validate contro i limiti stanza)
- `enable_terminal_input`: bool, thread stdin interno (disattivato sotto `ros2 launch`)

Si iscrive a

- `/vicon/tello_42_boosted/tello_42_boosted` → `pose_cb` (posa Tello, per hover/auto-advance)
- `/vicon/aruco42/aruco42` → `aruco_cb` (posa marker, per `target_mode=aruco_target`)
- `/tello/flight_state` → `flight_state_cb` (cattura hover al fronte di salita)
- `/target_handler/advance` → `advance_topic_cb` (avanzamento manuale da `mission_console`)

Pubblica

- `targets`
 - 25 Hz, sempre (anche a target invariato)
- `/tello/land_request`
 - solo quando `num_queues` è raggiunto

Espone

- `/target_handler/set_parameters`
 - (servizio, automatico)

observation_handler `tello_pkg`

Responsabilità: tutta la lettura Vicon, trasformate/quaternioni, costruzione dello spazio delle osservazioni (52 elementi). Non tocca mai `djitellopy`.

Parametri

- `dof_mask_mode`: full | uniciclo (default full)

Si iscrive a

- `/vicon/tello_42_boosted/tello_42_boosted` → `pose_cb` (input primario: quaternione → Eulero, derivata numerica → velocità angolare corpo, filtro passa-basso su lin/ang vel, gravità proiettata, sanity check NaN/Inf/quaternion degenera/jump implausibile)
- `targets` → `targets_cb` (coda waypoint da `target_handler`; azzerà l'integrale SOLO al cambio di `wp_idx`)
- `/tello/policy_action` → `policy_action_cb` (ultima azione, usata come `prev_action` nell'obs)

Pubblica

- `observations`
 - 52 elementi, 25 Hz (solo se pose fresca e targets ricevuti)
- `/tello/land_request`
 - se Vicon assente >3s consecutivi

Espone

- `/observation_handler/set_parameters`
 - (servizio, automatico)

Layout vettore osservazione (52 elementi)

`pos_env(3) · sin/cos yaw(2) · lin_vel_b(3) · ang_vel_b(3) · proj_grav_b(3) · prev_action(4) · preview 4 waypoint(4×5=20, ciascuno: delta pos(3)+sin/cos delta yaw(2)) · clearance stanza(6, distanza dai 6 muri) · dof_mask(4) · integral_norm(4) = 52`

policy_handler tello_pkg

Responsabilità: esegue in inferenza la policy HL (rete addestrata in Isaac Lab). Puro calcolo, nessuna logica di volo, non tocca mai djitellopy.

Configurazione

- checkpoint: policy_pos_controller/2026-07-31_13-02-43_ppo_torch/checkpoints/best_agent.pt (override via env var POLICY_CKPT_PATH) — MLP 256-128-64 ELU, normalizzazione con running mean/var salvati nel checkpoint

Si iscrive a

- observations** → **obs_cb** (mantiene solo l'ultima osservazione ricevuta + il suo timestamp)

Pubblica

- /tello/policy_action**
— 25 Hz, SOLO se l'osservazione ha meno di 0.2s (5 cicli): oltre quella soglia smette di pubblicare, propagando a cascata la "fame di dati" fino a vel_command_handler.

vel_command_handler tello_pkg

Responsabilità: UNICO nodo con una connessione djitellopy reale al drone (porte fisse 8889/8890, una sola istanza alla volta). Nessun'altra logica di calcolo/percezione.

Parametri

- enable_terminal_input: bool (thread stdin per 'start'/'q' da ros2 run standalone)

Si iscrive a

- /tello/policy_action** → **policy_action_cb** — SOLO se flight_ready: clip [-1,1] → **xVEL_REF_SCALE**([1,1,1,1.5]) → **cap sicurezza** (MAX_LIN_VEL_MPS=0.8, MAX_YAW_RATE_RADPS=1.0) → **x100** → **send_rc_control**. **NESSUN mascheramento dof_mask** qui: è solo una feature di osservazione, mai un hard mask sull'azione.
- /tello/land_request** → **land_request_cb** — atterraggio d'emergenza da nodi a valle
- /tello/start_request** → **start_request_cb** — **CANCELLO DI PARTENZA**: unico modo per far decollare il drone

Pubblica

- /tello/flight_state**

Sicurezza

- watchdog azione scaduta (>0.2s → stick a zero)
- failsafe batteria critica (<15% → land automatico)
- escalation telemetria drone persa (>3s → land automatico)
- Ctrl+C gestito con handler SIGINT custom: atterra SEMPRE, anche se non mai decollato
- takeoff/land via djitellopy con assestamento pre/post (2s / 3s), conferma quota (>15cm entro 8s, altrimenti considerato comunque in volo per sicurezza)

mission_console tello_pkg

Responsabilità: punto di controllo interattivo da terminale. Va SEMPRE lanciato a parte (secondo terminale): ros2 launch non inoltra lo stdin ai processi figli.

All'avvio

Wizard sequenziale bloccante: chiede target_mode, advance_mode, dof_mask_mode (+ coordinate custom se target_mode=custom), num_queues — applicati subito via set_parameters.

Nel loop operativo

- INVIO → pubblica **/target_handler/advance**
- I / land → pubblica **/tello/land_request**
- start / s → pubblica **/tello/start_request**

Client dei servizi

- /target_handler/set_parameters**
- /observation_handler/set_parameters**

6. Nodo della pipeline modulare — tello_pkg_web (versione con interfaccia web)

I nodi target_handler, observation_handler, policy_handler sono **riusati identici** da tello_pkg (nessuna copia, nessuna modifica): il launch web_pipeline.launch.py li avvia dal package tello_pkg. L'unico nodo nuovo è quello di attuazione/interfaccia.

vel_command_handler_web tello_pkg_web

Responsabilità: fonde TRE ruoli nello stesso processo (possibile solo perché il server web, a differenza dello stdin, non ha il problema dell'inoltro sotto ros2 launch):

- 1. **Attuazione/telemetria/watchdog djittelopy** — identica a vel_command_handler.py (stessi topic, stesse costanti, stesso gate esplicito su start).
- 2. **Ruolo "mission console" via HTTP** — stessi client SetParameters verso target_handler/ observation_handler, esposti come endpoint REST invece che da wizard testuale.
- 3. **Stato per la dashboard 3D** — sottoscrizioni PROPRIE (indipendenti da observation_handler, solo per visualizzazione, mai per il controllo) a Vicon/ArUco/targets.

Parametri

- enable_terminal_input, output_dir, save_csv, save_plot

Si iscrive a (attuazione — invariato da vel_command_handler)

- /tello/policy_action
- /tello/land_request
- /tello/start_request

Si iscrive a (SOLO dashboard, nessun impatto sul controllo)

- /vicon/tello_42_boosted/tello_42_boosted → pose_cb proprio: calcola pos/roll/pitch/yaw/lin_vel_b/ang_vel_b riusando le stesse funzioni pure importate da observation_handler.py (nessuna riscrittura della matematica)
- /vicon/aruco42/aruco42 → aruco_cb: marker ArUco SEMPRE visibile in scena, indipendentemente dal target_mode attivo
- targets → targets_cb: wp_idx/coda/target attivo per il pallino 3D

Pubblica

- /tello/flight_state
- /target_handler/advance
— da endpoint /api/advance

Client dei servizi

- /target_handler/set_parameters
— usato da /api/params e /api/custom_target
- /observation_handler/set_parameters
— usato da /api/params

Server HTTP/WebSocket (FastAPI + uvicorn, thread separato, porta 8080)

Endpoint	Metodo	Frequenza / trigger	Cosa fa
/static/*	GET	on-demand	serve index.html/app.js/style.css/vendor (three.js)
/api/status	GET	on-demand	ultimo snapshot di stato (stesso payload del WS)
/api/params	POST	submit form	imposta dof_mask_mode/target_mode/advance_mode/num_queues, SOLO se sessione idle
/api/custom_target	POST	click sulla scena 3D	imposta custom_target_x/y/z, validato contro i limiti stanza, applicabile in QUALSIASI momento (semantica runtime di target_handler)
/api/start	POST	bottone "Start algorithm"	chiama start_request_cb direttamente (gate esplicito: se non premuto, il drone non si muove)
/api/land	POST	bottone "Land"	emergency_land() incondizionato
/api/advance	POST	bottone "Next waypoint"	pubblica /target_handler/advance
/ws/state	WebSocket	10 Hz	broadcast continuo dello stato: posizione/assetto/velocità drone, target, marker ArUco, custom_target, batteria, coda, parametri correnti

Registrazione dati

Alla chiusura del nodo (Ctrl+C): CSV (sensor_vicon_*.csv, sensor_tello_flight_*.csv) + PNG con griglia 4x2 (posizione, traiettoria 3D, angoli Eulero, gravità proiettata, velocità lineari, ang_vel_b, istogramma ang_vel_b, batteria/quota) — stessa logica di position_controller_web.py.

Gate di sicurezza invariato: in entrambe le versioni il drone non si muove finché non viene premuto "Start" — flight_ready resta False finché start_request_cb/il tasto in dashboard non lo marca True; policy_action_cb ignora ogni azione ricevuta prima di quel momento.

7. Nodi legacy monolitici (non toccati, ancora eseguibili)

`position_controller_VICON_VERSION.py` `tello_pkg` e `position_controller_web.py` `tello_pkg_web` sono i predecessori monolitici della pipeline modulare: un solo processo che fa TUTTO (lettura Vicon, calcolo osservazioni, inferenza policy, generazione target, attuazione djitellopy — e per la versione web, anche il server FastAPI). Timer unico a 25 Hz (STEP_DT=0.04s) per l'intero control_loop. Non fanno parte della pipeline modulare descritta sopra, ma restano negli entry_points come alternativa "tutto-in-uno" già validata.

8. Nodi standalone di utilità — tello_test

Nodo	Si iscrive a	Frequenze	Scopo
takeoff_land.py	/vicon/<pose_topic> (param.)	telemetry 10 Hz (0.1s) · status 0.5 Hz (2s) · rc resend test 10 Hz (0.1s)	Test isolato: connessione djitellopy, takeoff, comando rc di test, land, log/plot CSV. Non fa parte della pipeline (non lancia policy).
read_sensors.py	/vicon/<pose_topic> (param.)	telemetry 10 Hz (0.1s) · status 2 Hz (0.5s)	SOLO monitoraggio: si connette a djitellopy solo per leggere batteria/quota (nessun comando di volo), logga Vicon + telemetria su CSV/plot.

Nota: takeoff_land.py e read_sensors.py usano la stessa libreria djitellopy e quindi non possono girare in parallelo con nessuno dei nodi di attuazione della pipeline (porte UDP 8889/8890 fisse per processo).

9. Riferimenti rapidi — limiti stanza e soglie di sicurezza

Costante	Valore	Definita in	Uso
ROOM_MIN	[-2.0, -1.5, 0.1] m	target_handler / observation_handler	limite inferiore stanza (x,y,z)
ROOM_MAX	[2.0, 1.5, 2.0] m	target_handler / observation_handler	limite superiore stanza (x,y,z)
TARGET_ROOM_MARGIN	0.8	target_handler	marginare dai muri per target random (singolo/variabile)
MAX_LIN_VEL_MPS	0.8 m/s	vel_command_handler_web	cap di sicurezza velocità lineare
MAX_YAW_RATE_RADPS	1.0 rad/s	vel_command_handler_web	cap di sicurezza velocità di imbardata
BATTERY_FAILSAFE_PCT	15 %	vel_command_handler_web	soglia batteria critica → land automatico
TARGET_REACH_THRESHOLD_M	0.15 m	target_handler	distanza per considerare il target raggiunto (advance_mode=auto)
TARGET_HOLD_TIME_S	1.2 s	target_handler	tempo di permanenza richiesto entro soglia prima di avanzare